IUT Grand Ouest
Sciences des Données
1ère année de STID

SAE 2.01
Conception et implémentation d'une base de données

Auteur
Thibaut Dupui

Année universitaire : 2022 / 2023

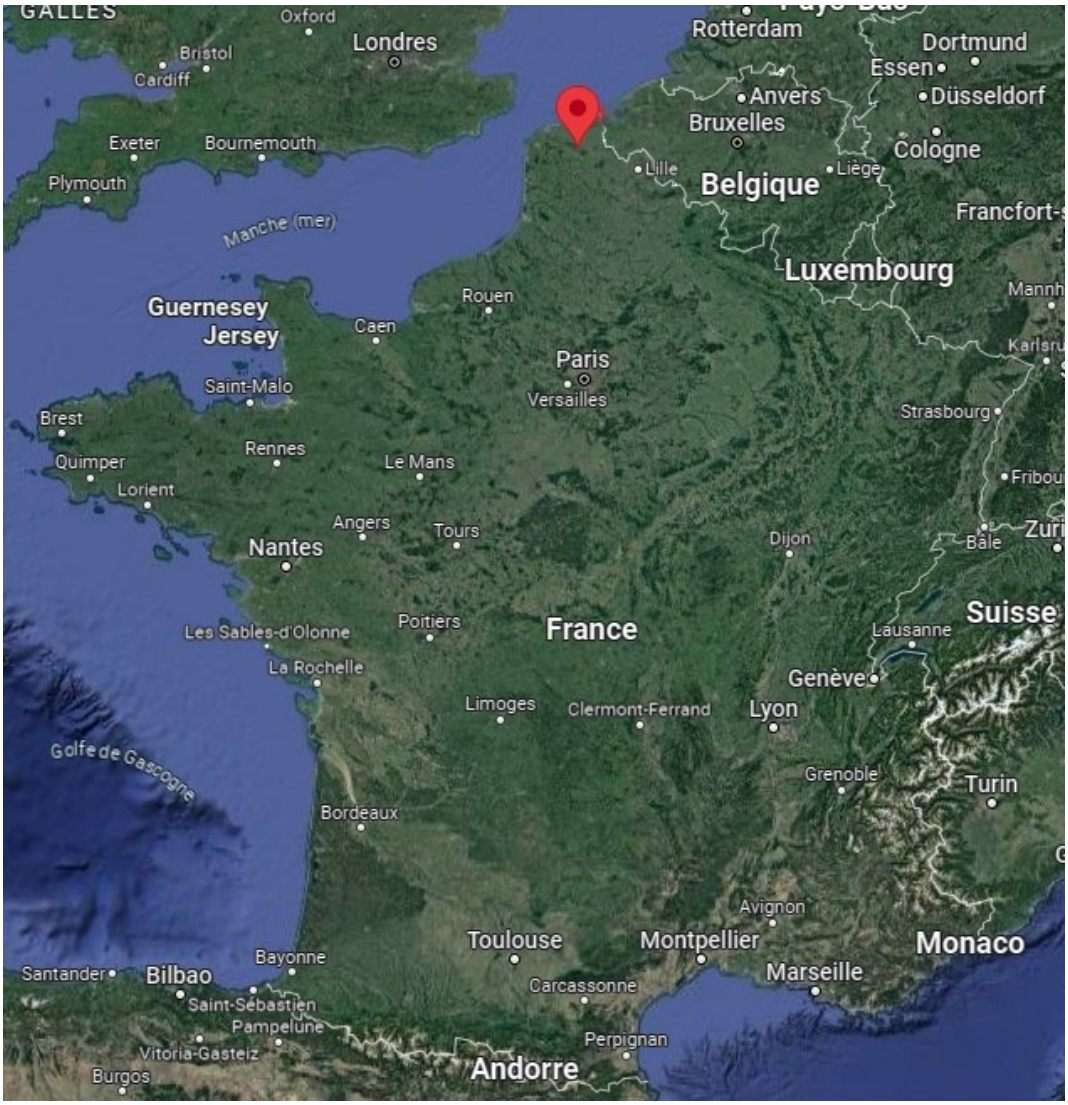
Table des matières

		Page
0.1	Introduction	3
0.2	Problématique	4
0.3	Organisation	5
0.4	Explication des différentes parties du projet	6
0.5	Résolution	7
0.6	Difficulté	10
0.7	Conclusion	11

0.1 Introduction

Nous disposons de la liste de tout **les produit d'un magasin Proxi**.
Le but est de mettre en place une **base de données** pour palier à la **coupure internet** de l'établissement qui du coup n'est plus en capacité d'utiliser les outils de outils de la centrale de Paris.

Le magasin Proxi est situer dans une petite commune du nord, à **Wulverdinghe**, où habite 322 habitants. nous devons donc développer des outils en local pour permettre au habitant de continuer a faire leurs courses dans de bonne conditions.



— 0.2 Problématique

Dans un premier temps il faut créé une base de donné avec les donner insérer dans cette dernière. De plus cette base de données devra être présenter sous forme d'UML Ensuite dans une seconde partie de la SAE, nous devons **développez un outil en ligne de commande** avec **4 différent thèmes** :

— **Rentrer un produit dans le stock** :

- Afin d'enregistrer les produit informatiquement dans le stock du magasin

— **Ranger un produit** :

- Pour enregistrer les produits qui sont placé dans les rayons

— **Retirer un produit du stock**

- Enregistré le retrait d'un produit issue du stock.

— **Rechercher un produit dans le stock**

- Rechercher un produit grâce a son code, qui est unique pour chacun.

Nous devrons avec ceci afficher les **logs** retraçant toutes ces opérations

Après sa nous devons **compléter une interface graphique** pour avoir un compte rendus des transactions effectuer entre l'utilisateur et les clients, c'est à dire **quelle article a été acheter, son prix et la quantité** afin de gérer les article encore **disponible en magasin et dans le stock**.

Enfin il faudra **créé un outils permettant d'analyser** les ventes sous forme de **graphique** pour en tirer d'éventuelles information.

— 0.3 Organisation

Nous somme, pour cette SAE un groupe de 5 :

Thibaut : Chargé de de rédiger **tout** le rapport sous LaTeX .

Matthias : S'occupe de **toute** la partie codage et programmation.

Yolan : A essayé.

Thomas : Présent physiquement.

Sarah : Absente.

— 0.4 Explication des différentes parties du projet

Pour ce faire nous allons **suivre plusieurs étapes** qui seront énumérer au fur et a mesure du rapport dans les différentes parties.

— **Partie A : conception de base de données** :

- Concevoir une base de données Sqlite documenté permettant de classer les articles par type et où ils sont dans le magasin et au format UML

— **Partie B : Stockages** :

- Développer un outil pour que les utilisateurs puisse gérer le stock en fonction des achats et de l'approvisionnement.

— **Partie C : Vente** :

- Compléter une interface graphique afin de pouvoir encaisser les clients et sortir un ticket.

— **Partie D : Compte Rendu** :

- Analyser les ventes a l'aide de graphique afin d'en tirer d'éventuelle conclusion.

— 0.5 Résolution

Dans cette partie, nous aborderons la **résolution du problème** lié à la gestion du stock dans notre application. Nous présenterons notre **choix** du système de gestion de base de données (SGBD) SQLite, ainsi que la **création de la table "Stock"** contenant tous les articles. De plus, nous détaillerons les différentes fonctionnalités mises en place pour permettre à l'utilisateur de **gérer efficacement** le stock en ligne de commande.

Choix du SGBD SQLite : Pour répondre aux besoins de notre application de gestion de stock, nous avons opté pour le système de gestion de **base de données SQLite**. Les raisons de ce choix résident dans la **simplicité d'installation**, la **légèreté** et la **portabilité** de SQLite, ce qui en fait un outil idéal pour notre application.

Création de la table "Stock" : Afin de **stocker les informations** relatives aux articles, nous avons **créé une table appelée "Stock"** dans notre base de données SQLite. Cette table est destinée à contenir **toutes les données** nécessaires à la gestion du stock, telles que le **nom** de l'article, sa **quantité** disponible, son **emplacement** dans le magasin, etc. Nous avons défini les différents champs de la table en fonction des besoins spécifiques de notre application.

Fonctionnalités de gestion du stock en ligne de commande : Pour permettre à l'utilisateur **d'interagir** avec le stock, nous avons développé des fonctionnalités accessibles en **ligne de commande**. Ces fonctionnalités offrent à l'utilisateur la possibilité de réaliser les actions suivantes :

a. Rentrer un produit dans le stock : L'utilisateur peut **ajouter un nouveau produit** au stock en fournissant les informations requises, telles que le nom de l'article, sa quantité, son emplacement, etc. Lorsque les données sont saisies, elles sont insérées dans la table "Stock" de la base de données.

b. Ranger un produit : Cette fonctionnalité permet à l'utilisateur de **modifier l'emplacement d'un produit** déjà présent dans le stock en le **rangeant** soit du magasin vers le stock, soit du stock vers le magasin. L'utilisateur peut spécifier le produit à ranger ainsi que l'emplacement de destination. L'opération de rangement consiste à **mettre à jour** les données dans les tables "Magasins" et "Stock" en conséquence, permettant ainsi de refléter le nouvel emplacement du produit. En fonction de la direction du rangement, les données sont transférées entre les tables "Magasins" et "Stock" pour assurer une gestion précise des emplacements des produits.

c. Retirer un produit du stock : L'utilisateur peut **retirer un produit** spécifique du stock en indiquant son nom ou un identifiant unique. Une fois que le produit est supprimé de la table "Stock", il n'est plus pris en compte dans les opérations de gestion du stock.

d. Rechercher un produit dans le stock : Cette fonctionnalité permet à l'utilisateur de **rechercher un produit** dans le stock en fournissant son nom ou d'autres critères de

recherche. L'application effectue une requête dans la table "Stock" et affiche les résultats correspondants.

Pour assurer une meilleure traçabilité des articles et des actions effectuées, il est prévu que des **logs** soient générés pour chaque action réalisée dans le système de gestion du stock. Ces logs enregistreront les détails pertinents tels que **la date et l'heure de l'action**, l'utilisateur **responsable de l'action**, ainsi que les informations spécifiques à **l'action effectuée** (par exemple, le produit ajouté, le produit rangé ou retiré, les critères de recherche, etc...).

Dans le cadre de ce projet, nous avons pris en charge **la partie "Vente"** en complétant l'interface graphique fournie à l'aide de la **bibliothèque PyQt5**. Cette interface offre aux utilisateurs la possibilité **d'encaisser les clients** de manière simple et efficace.

L'une des fonctionnalités clés que nous avons développées est la capacité d'encaisser les articles en **scannant** leur code **EAN**. Une fois que l'utilisateur entre le code, le système enregistre automatiquement la transaction et **génère un ticket** de caisse au **format TXT**. Ce ticket de caisse contient toutes les **informations pertinentes**, telles que les **articles achetés**, les **quantités**, les **prix unitaires**, et le **montant total** de l'achat.

Afin de garantir la **traçabilité** des transactions et de faciliter les **futures analyses**, nous avons mis en place un **système de sauvegarde** des tickets de caisse dans une **base de données**. Ainsi, chaque ticket de caisse enregistré est stocké de manière sécurisée, permettant une analyse ultérieure des ventes réalisées.

L'utilisation de cette fonctionnalité de sauvegarde dans une base de données offre de nombreux avantages. Elle permet, par exemple, de **générer des rapports** détaillés sur les ventes **quotidiennes**, **hebdomadaires** ou **mensuelles**. Ces rapports peuvent être utilisés pour évaluer les **performances de vente**, identifier les **articles populaires** ou les **tendances d'achat**, et prendre des décisions stratégiques éclairées pour le **développement du commerce**.

En résumé, grâce à l'intégration de la bibliothèque PyQt5, nous avons **enrichi l'interface graphique** en permettant aux utilisateurs d'encaisser les clients en saisissant le **code EAN** des articles. Les transactions sont enregistrées dans des tickets de caisse générés au format TXT, puis sauvegardées dans une base de données pour des fins d'analyse ultérieure. Cette fonctionnalité facilite la gestion des ventes et fournit des informations précieuses pour orienter les décisions commerciales.

Pour finir cette partie Résolution, nous avons créé un **outil d'analyse** des ventes utilisant un **Dashboard**, en utilisant les **bibliothèques Matplotlib et Seaborn**. Cette étape finale nous a permis de visualiser de manière **claire et concise** les données relatives aux ventes effectuées.

Le Dashboard comprend **plusieurs graphiques**, dont deux **graphiques à barres** essentiels pour notre analyse. Le **premier graphique** représente le **total des ventes par jour**, ce qui permet de visualiser les **variations quotidiennes** des ventes. Cela nous offre des informations précieuses sur les **périodes de forte activité** commerciale. Le **deuxième graphique** à barres représente le **montant moyen des ventes par mois**, nous donnant ainsi une vue d'ensemble de la performance des ventes sur une **base mensuelle**.

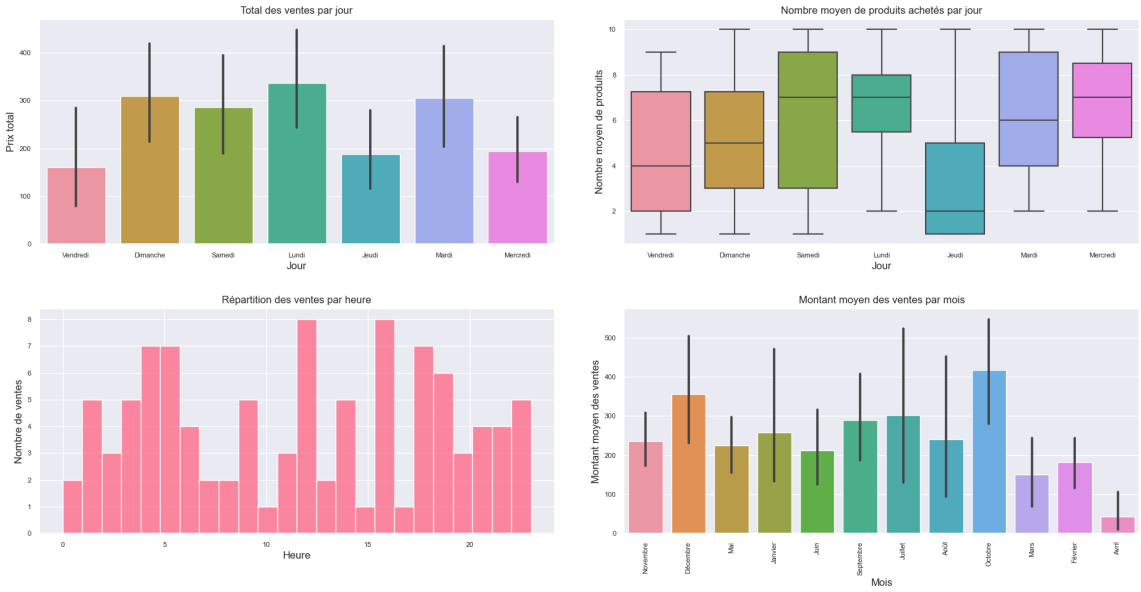
En complément de ces graphiques, nous avons également inclus un **histogramme** qui présente la **répartition des ventes par heure**. Cette visualisation permet de **mettre en évi-**

dence les heures de la journée où les ventes sont les plus élevées, ce qui peut être utile pour la planification des ressources et des horaires de travail.

Enfin, pour une analyse plus approfondie, nous avons inclus un **graphique à boîtes à moustaches** représentant le **nombre moyen de produits achetés par jour**. Ce graphique nous permet de mieux comprendre la **variation de la demande** des clients sur une base quotidienne, en identifiant les jours où les achats sont plus importants ou plus faibles.

L'utilisation des bibliothèques Matplotlib et Seaborn pour créer ce **Dashboard d'analyse** des ventes a permis d'obtenir des visualisations **claires et informatives**. Ces graphiques offrent une **représentation visuelle puissante** des données, facilitant ainsi l'interprétation des tendances et des modèles de vente. Grâce à cet outil d'analyse, les décideurs auront une **meilleure compréhension** de la performance commerciale, ce qui les aidera à **prendre des décisions** stratégiques éclairées pour **l'avenir de l'entreprise**.

Voici maintenant un exemple du dashboard :



— 0.6 Difficulté

La réalisation de ce projet a été ponctuée par **plusieurs difficultés** auxquelles nous avons dû faire face et trouver des **solutions appropriées**.

La **première difficulté** majeure rencontrée a été **l'insertion des données** dans la base de données SQLite dédiée au stockage des informations. Cette partie du projet, intitulée **"Stockage"**, a requis une compréhension approfondie du fonctionnement de SQLite ainsi que la mise en place de requêtes adéquates pour l'insertion des données.

La **deuxième difficulté** a été liée à la création de **l'interface graphique** représentant une caisse de magasin. Dans cette partie du projet, appelée **"Vente"**, les utilisateurs devaient entrer un code article appelé EAN. La conception de cette interface graphique a exigé une attention particulière pour garantir une expérience utilisateur **fluide et intuitive**. Néanmoins, nous avons dû surmonter des **défis d'implémentation** et de validation des données, nécessitant des **ajustements** et des **itérations** supplémentaires.

La **troisième difficulté** majeure a été la **création du dashboard** à l'aide des bibliothèques Matplotlib et Seaborn. Ces librairies sont largement utilisées pour la visualisation de données en Python. Cependant, l'une des difficultés rencontrées concernait **l'ordre incorrect des mois** dans le **deuxième graphique** du dashboard. Malheureusement, malgré nos **multiples tentatives** pour résoudre ce problème, nous n'avons **pas réussi** à le résoudre dans le cadre de ce projet.

En conclusion, malgré les difficultés rencontrées dans ce projet, **nous avons su relever les défis** et trouver des solutions pour avancer. Nous reconnaissons la persistance de la difficulté liée à l'ordre incorrect des mois dans le deuxième graphique du dashboard, et nous sommes **conscients de son impact** sur l'analyse des données. **Cependant**, nous considérons cette expérience comme une occasion **d'apprentissage** et de **croissance**, nous incitant à approfondir notre compréhension des outils utilisés et à chercher des solutions plus adaptées dans de **futurs projets**.

0.7 Conclusion

Ce projet de **conception et d'implémentation d'une base de données** pour le magasin **Proxi** à Wulverdinghe a été un défi **passionnant et enrichissant**. Notre **objectif était de fournir une solution** locale permettant aux habitants de continuer à faire leurs courses dans de bonnes conditions malgré **la coupure Internet** de l'établissement. Grâce à notre travail, nous avons réussi à mettre en place un **système robuste et fonctionnel** qui répond aux besoins du magasin et de ses clients.

Dans **la première partie** de notre projet, nous avons **conçu et documenté** une base de données SQLite pour **classer les articles** par type et gérer leur emplacement dans le magasin. Nous avons utilisé le **langage UML** (Unified Modeling Language) pour modéliser la structure de la base de données, ce qui nous a permis de visualiser clairement les relations entre les différentes entités.

Ensuite, nous avons **développé** un **outil en ligne de commande** avec **quatre fonctionnalités** principales : **l'ajout** de produits au stock, **le rangement** des produits dans les rayons, **le retrait** de produits du stock et **la recherche** de produits. Ces fonctionnalités ont permis aux utilisateurs de **gérer efficacement** le stock du magasin, d'assurer une **traçabilité** des actions effectuées et de faciliter la recherche d'articles spécifiques.

Nous avons également complété une **interface graphique** pour encaisser les clients et **générer des tickets** de caisse détaillés. Cette interface a facilité les transactions avec les clients, en particulier grâce à la possibilité de scanner les **codes EAN** des articles. Les **tickets** de caisse ont été **enregistrés** dans une **base de données** pour permettre une **analyse ultérieure** des ventes et une gestion précise des stocks.

Enfin, nous avons **créé** un **outil d'analyse des ventes** sous forme d'un **dashboard** utilisant les bibliothèques Matplotlib et Seaborn. Malgré la difficulté rencontrée avec **l'ordre incorrect des mois** dans l'un des graphiques, nous avons **réussi** à **visualiser les données** de manière **claire et concise**, fournissant ainsi aux décideurs des informations précieuses sur les performances de vente, les tendances et les modèles.

En conclusion, ce projet nous a permis **d'acquérir** de nouvelles **compétences** en matière de conception de bases de données, de développement d'outils en ligne de commande et d'interfaces graphiques, ainsi que d'analyse et de visualisation des données. Malgré les difficultés rencontrées, **nous sommes fiers** du résultat obtenu et de la contribution que notre solution apporte à la communauté locale de Wulverdinghe. Nous espérons que notre travail aidera le magasin Proxi à continuer à servir ses clients de manière efficace et à prendre des décisions éclairées pour le développement de leur entreprise.